Read the question, name its shape, open that band, find the tag. Side 1: bands 0, 1, 2 (Q1 to Q8). Side 2: band 2 (Q9 to Q16), bands 3, 4, 5.

1 · "What is printed by this code?"
constructor and destructor lines → **Rule table** · by value, return by value → **Q20 shape** · two bases, virtual public, "why different" → **Q27 / Q28 shape** · slicing, virtual destructor, virtual function → **Parent / Child shape** · unknown word → band 0.

2 · "What is X? When is it used?"
Q1 copy ctor · **Q2** ref vs ptr · **Q3** value / ptr / ref · **Q4** &x *x · **Q5** static / dynamic · **Q6** leak · **Q7** const · **Q8** dtor when · **Q9** init ×3 · **Q10** :: vs . · **Q11** class / struct · **Q12** forward decl · **Q13** abstract · **Q14** interface · **Q15** diamond · **Q16** copy-swap. Three moves: **Name**, **Contrast**, **Code**, plus **Used when** if asked.

3 · "Write the function / class"
Fraction class, throwing constructor, copy constructor, destructor, <<, >>, file-reading main → **Fraction** · +, <, own exception type → the notes under it · class owning an array → **Deep copy** · abstract class and child → **Abstract class** .

4 · True/false, multiple choice
Exceptions · **STL** · **Static, elision** · **Right-left rule** · **Weeks 1 and 2**. Band **5**: **Design** (SOLID, Demeter) · **Operators and friends** · **Classes and inheritance** · **UML**.
yellow the paper's words · blue plain English · **red** trap · green code

0. Reading C++: the words every question uses

A() {} **constructor** → runs once when an object is **born**; A(int n) takes a value

A(const A& o) **copy constructor** → born as a **photocopy** of object o

~A() {} **destructor** → runs once when it **dies**: the } ending its block, or delete

A(int x) : n(x) {} → **initializer list**: member n set to x before the body

int& r = x; **reference** → a second name for x; change r and x changes

int* p = &x; **pointer** → the **address** of x; *p = what it points at; p->f() calls f on it

1. "What is printed by this code?"

Rule table: read main top to bottom; at each line ask who is born, who is copied, who dies; count against the HINT

A a; → Default constructor now; Destructor at the block's }

A b = a; or A b(a); → Copy constructor now; Destructor at }

b = a; (b exists already) → **nothing**: assignment is not a birth

f(a) with f(A x) (by value) → Copy constructor (x); x's Destructor when f ends

f(a) with f(A& x) or f(const A& x) → **nothing**: no copy, no death

in f: return x; (x the **parameter**) → Copy constructor (the result); an ignored result dies at that line's end, then x dies

A y = f(a); → the result **is** y: no extra copy; y dies at }

in f: return A(..); or return local; → no copy (elided)

D d; with D : public B → B constructor, then D; dies D, then B (reverse)

E : public C, public D → C's chain then D's (order written); dies D's then C's

B : virtual public A → the one shared A: once, first; dies once, last

B* p = new D; delete p; → B, D born; dies D then B if ~B virtual, **else B only**

B b = d; or f(B x) given a D → **slicing**: B's copy constructor only; dies B only

block end } → locals die in **reverse** order of birth

Q20 shape **A foo(A input) { print "Look ma"; return input; }, hint 14 lines**

A littleA; → 1 Default constructor!

foo(littleA); result unused → 2 Copy constructor! input · 3 Look ma, I'm fooing! · 4 Copy constructor! result · 5 Destructor! result · 6 Destructor! input

A anotherA = littleA; → 7 Copy constructor!

A yetAnotherA = foo(littleA); → 8 Copy constructor! input · 9 Look ma, I'm fooing! · 10 Copy constructor! result = yetAnotherA · 11 Destructor! input

end of main → 12, 13, 14 Destructor! yetAnotherA, anotherA, littleA

Tally 1 default, 5 copy, 2 fooing, 6 destructor. If foo took A& input: lines 2, 6, 8, 11 vanish (10 lines); the result is still copied.

Q27 / Q28 shape **E : public C, public D; C and D from B; B from A; E e;**

no virtual (14 lines) → born **A B C A B D E**, dies **E D B A C B A**

B : virtual public A, C : virtual public B (12 lines) → born **A B C B D E**, dies **E D B C B A**

No virtual: two full A→B chains (under C, under D), death in exact reverse. **"Why different"**: with virtual, the one shared A is built once, first, by E; C's B is virtual but D's B is an ordinary base built again, so B prints twice and A once.

Parent / Child shape **Parent/Child, lines "X default constructor." "A Parent copy was made." "X destructor."**

Child c; → Parent default constructor. · Child default constructor. at }: Child destructor. · Parent destructor.

passByValue(c); with (Parent p) → A Parent copy was made. (a Child squeezed into a Parent box: slicing) · in passByValue · Parent destructor.

passByReference(c); with (Parent& p) → in passByReference (no copy, no death)

Parent* p = new Child; delete p; → Parent default constructor. · Child default constructor. · Child destructor. · Parent destructor.

same, ~Parent() **not** virtual → … Child default constructor. · **Parent destructor. only**: Child's never runs ("BAD")

Animal* p = new Bird; p->speak(); → speak not virtual: Animal's "???"; virtual: Bird's "Chirp". Bird b; b.speak(); always Chirp

friend void f(MyClass mc) { mc.x++; cout << mc.x; } → prints 1; the caller's object is unchanged (a copy)

2. "What is X? How is it different from Y? When is it used?"

Q1 Copy constructor (when is it used?)

Name: builds a **new object as a copy of an existing one**. & because a by-value parameter would itself need a copy (endless); const so const objects can be copied. **Contrast:** **not** b = a; on an existing b (assignment). The compiler's version is **shallow** (copies pointers, not what they point at); write your own for a **deep** copy. **Code:** A(const A& other)
Used when: a new object starts from another: A b(a); A b = a;, pass by value, return by value.

Q2 Reference vs pointer

Name: a **reference** is an **alias**, "a nickname for an existing variable": must be initialized at declaration, never re-seated. **Contrast:** a **pointer** holds an **address**: may be null, may be re-pointed, needs *p to reach the value. Use a reference to change the caller's variable, to avoid a copy, or for polymorphism. **Code:** int& r = n; · int* p = &n;

Q3 Pass by value / pointer / reference (why choose one?)

Name: value = a copy; **pointer** = the address; **reference** = the variable itself under a new name. **Contrast:** value: the caller's variable cannot change. Pointer: *p = 5 changes it; may be nullptr. Reference: changes it, no copy, never null. **Code:** f(int x) · f(int* p) · f(int& r) **Used when:** value for small types you will not change; pointer when "none" (nullptr) is possible; reference to change it or skip a copy; const T& = no copy and no change.

Q4 &success vs *success (when would we use each?)

Name: *success is the **string itself** (dereference). **Contrast:** &success is the **address of the pointer variable** (string**). **Code:** string* success = &s; *success = "done"; · void reset(string** pp) { *pp = &other; } **Used when:** *success to read or change the string; &success so a function can re-point the pointer.

Q5 Static vs dynamic allocation

Name: static: on the **stack**, size fixed at compile time, dies by itself at }. **Contrast: dynamic**: on the **heap**, size chosen at run time, used through a pointer, lives until **you** delete. **Code:** int x = 5; A a(1); · int* p = new int(5); new int[n];

Q6 Memory leak (prevent it; underline the fix)

Name: heap memory from new never freed with delete: unreachable until the program ends. **Contrast:** the fix: pair every new with a delete (new[] with delete[]), free in the destructor. **Code:** void leak() { int* p = new int[100]; /* use */ delete[] p; }

Q7 const argument / const function

a. **Name:** a const argument: the function **promises not to modify** it. **Contrast:** with & it also avoids a copy. **Code:** void print(const A& a) b. **Name: const member function**: promises not to change any data member. **Contrast:** only these can be called on a const object or through const A&; getters get it. **Code:** int get() const;

Q8 Destructor runs automatically when?

Name: when the object **goes out of scope** (its block's }, also while an exception unwinds); a **temporary** at the end of its statement; members and bases after the owner's destructor body. Reverse order of construction. **Contrast:** a heap object: only on delete (delete[] for arrays). **Code:** { A a; } dies at } · A* p = new A; delete p;

2. continued: definitions

Q9 Three ways to initialize; preferred?

Name: =, parentheses, **braces**; prefer **braces**. **Contrast:** braces refuse **narrowing** where the other two silently truncate; they work for every type. **Code:** int x = 0; · int x(0); · int x{0}; · int y{3.7}; will not compile

Q10 :: vs .

Name: :: **scope resolution**: a name inside a namespace or class. **Contrast:** . a **member of an object**; -> the same via a pointer. **Code:** std::cout · A::A(int) defined outside · A::count() · a.f()

Q11 class vs struct

Name: only the **default access** differs; otherwise identical. **Contrast:** struct public, class private (also for inheritance). **Code:** struct P { int x; }; x public · class Q { int x; }; x private

Q12 Forward declaration (and its purpose)

Name: a name declared before its definition, so code can use Car* or Car& before Car is defined (classes referring to each other, friend class Car;); fewer includes, no circular includes. **Contrast:** **cannot** create a Car or use its members yet (incomplete type). **Code:** class Car;

Q13 Make a class abstract

Name: at least one **pure virtual** function. No abstract keyword. **Contrast:** **cannot be instantiated**; a child stays abstract until it overrides every pure virtual. **Code:** virtual double area() const = 0;

Q14 Does C++ have interfaces?

Name: no keyword: a **pure abstract class**. **Contrast:** only pure virtuals, no data, a virtual destructor; implemented by inheriting (multiple allowed) and overriding all. **Code:** class I { public: virtual void f() = 0; virtual ~I() {} };

Q15 Diamond: how, ambiguity, most-derived

a. **Name:** student and mathematician both inherit person, and math_student inherits both. **Contrast:** person appears **twice** unless the middle classes inherit virtual. **Code:** class student : public virtual person b. **Name:** a function on both sides (all_info) is **ambiguous**. **Contrast:** bob.all_info() will not compile: qualify, override it in math_student, or use virtual inheritance. **Code:** bob.mathematician::all_info() c. **Name:** with virtual bases the **most-derived class must call the shared base's constructor itself**. **Contrast:** the middle classes' person(..) calls are ignored; forget it and person is default-built. **Code:** math_student(..) : person(name), student(..), mathematician(..)

Q16 Copy and swap: how, easy error

Name: 1. parameter **by value**, so the copy constructor makes the copy; 2. swap every member with it; 3. the copy, holding the old state, dies at return and frees it. **Contrast:** the **errors**: parameter by reference (no copy, you swap the caller away); std::swap on the whole object (calls = again: endless); a forgotten member. **Code:** A& operator=(A other) { mySwap(*this, other); return *this; }

3. "Write the function / class / program"

Fraction Q21 to Q26: the class, a file into a stack, print

```
#include <iostream>  also fstream, stack, stdexcept
using namespace std;
class Fraction {
private: int numerator; int denominator;
public:
  Fraction(int n, int d)  constructor
    : numerator(n), denominator(d) {  initializer list
    if (d == 0) throw invalid_argument(
      "Denominator cannot be zero"); }
  Fraction(const Fraction& o)  copy constructor: const + &
    : numerator(o.numerator),
      denominator(o.denominator) {}
  ~Fraction() {}  destructor: nothing to free
  friend ostream& operator<<(ostream& os,  print
      const Fraction& f) {
    os << f.numerator << "/" << f.denominator;
    return os; }  must return os
  friend istream& operator>>(istream& is,  read
      Fraction& f) {  not const: it changes f
    is >> f.numerator >> f.denominator;
    return is; }
  friend bool operator<(const Fraction& a,
      const Fraction& b) {  lets sort work
    return a.numerator * b.denominator
        < b.numerator * a.denominator; }
};
int main() {  file → stack → print; catch with catch (const
invalid_argument& e) { cout << e.what(); }
  ifstream in("Fractions.txt");
  if (!in.is_open()) {
    cerr << "Could not open" << endl; return 1; }
  stack<Fraction> st; int num, den;
  while (in >> num >> den)  until end of file
    st.push(Fraction(num, den));
  while (!st.empty()) {  last pushed, first out
    cout << st.top() << endl; st.pop(); }
}
```

operator+: return Fraction(a.n*b.d + b.n*a.d, a.d*b.d); (a friend, both const&)

```
class BadDenom : public invalid_argument {
public: BadDenom()  your own exception type
    : invalid_argument("Denominator cannot be zero") {} };
```

Deep copy a class owning an array

```
class Character { int* ids; int size; public:
  Character(int n) : ids(new int[n]{}), size(n) {}
  Character(const Character& o)  copy constructor
    : ids(new int[o.size]), size(o.size) {  new array
    for (int i = 0; i < size; ++i)  copy the contents
      ids[i] = o.ids[i]; }
  ~Character() { delete[] ids; } };  free what you own
```

Abstract class and its child

```
class Shape { public:
  virtual double area() const = 0;  pure virtual
  virtual ~Shape() {} };  Shape s; = error
class Circle : public Shape { double r; public:
  Circle(double r) : r(r) {}
  double area() const override { return 3.14 * r * r; } };
```

4. True/false and multiple choice traps

Exceptions: catch (const X& e) (by value slices) · first match wins: child before parent, catch(...) last · bare throw; rethrows · uncaught → std::terminate · a constructor that throws makes **no object**: no destructor for it, built bases and members are destroyed · destructors are implicitly noexcept, **never throw from one** · <stdexcept>: logic_error (invalid_argument, out_of_range, length_error), runtime_error (range_error, overflow_error).

STL: m[k] on a missing key **creates** it and m[k] = v overwrites; insert refuses a duplicate key · set skips equal elements; multiset::erase(key) removes all equal · ordering by operator<, a predicate or a functor (strict weak: irreflexive, antisymmetric, transitive) · priority_queue top = **largest** · sort needs random access (l.sort() for a list) · iterators: bidirectional list, map, set (++ -- only); random access vector, deque, array (it + n) · [first, last): end() is one past the last, never dereference it · v[i] unchecked, v.at(i) throws out_of_range · containers store **copies**; for (auto x : v) copies, auto& x modifies.

Static, elision: one copy per class, shared; **defined outside** int X::n = 0; (keyword on the declaration, not the definition) · plain static cannot be initialised inside the class; const static integral can; constexpr static must · static function: no this, **not virtual or const**, only static data; call X::f() · RVO: return Number(n); builds **one** object (C++17 guaranteed); NRVO return local; "no guarantee"; a returned **parameter** is never elided.

Right-left rule: start at the name, go **right** to) or ;, then **left** to (, exit the parentheses, repeat. int (**ppf)() ptr→ptr→function→int · int *(*pap)[] ptr→array of ptr→int.

Weeks 1 and 2: -7 / 2 = -3, -7 % 2 = -1 (sign of the left operand) · C c(); declares a **function**, C c{}; an object · int x{3.7} is a narrowing error, = and ( ) truncate · while (cin >> n) stops on bad input or EOF; after cin >> n the newline stays, so the next getline is empty.

5. On the slides, not on the practice paper

Design: SOLID · **S**ingle responsibility: one reason to change per class · **O**pen for extension (inherit, override), closed for modification · **L**iskov: a subtype stands in for its base without breaking the program · **I**nterface segregation: many client-specific interfaces, none forcing unused functions · **D**ependency inversion: depend on abstractions, not concrete classes · **Demeter**: talk only to friends, no a.getB().getC().f().

Operators and friends: unary (++ -- !) and binary that change the left operand (+= -= =) are **members**; binary that make a new value (+ - == !=) and << >> are **friendly non-members** with const T& parameters · a friend is declared inside the class but is not a member; friendship is **not mutual, not transitive, not inherited** · operator<< is never a member: its left operand is the stream.

Classes and inheritance: struct defaults public, class private, for inheritance too (class X : Spy is private) · public inheritance keeps public and protected; protected makes both protected; private makes both private; private base members are never accessible · members initialise in **declaration order**, not list order · any constructor you write removes the default one · not inherited: constructors, destructor, friends · delete basePtr; runs the derived destructor only if the base destructor is virtual.

UML: + public, # protected, - private; underlined static; *italic* abstract · generalization solid, realization dashed (hollow triangles at the base) · composition (owns-a, dies with the owner) filled diamond, aggregation (has-a, outlives it) hollow; dependency a dashed arrow, association a line.